

INFORMATION SECURITY PROJECT REPORT

SECURE VOTING SYSTEM



A Cryptographically Secure Electronic Voting Platform

Department: BSCS7A

Submitted By:

S3F22UBSCS092	Hassan Ali
S3F22UBSCS099	Moeed Amjad
S3F22UBSCS053	Ameer Hamza

Submitted To:

Ma'am Sheeza Shabbir

Academic Year: 2025-2026

Contents

- 1. INTRODUCTION 4
 - 1.1 Background 4
 - 1.2 Motivation..... 4
 - 1.3 Scope..... 4
- 2. OBJECTIVES 4
- 3. TOOLS AND TECHNOLOGIES USED 5
 - 3.1 Frontend Technologies 5
 - 3.1.1 HTML5 (HyperText Markup Language) 5
 - 3.1.2 CSS3 (Cascading Style Sheets)..... 6
 - 3.2 Backend Technologies 6
 - Flask (Python Web Framework) 6
 - 3.3 Database Technologies 6
 - SQLite 6
 - 3.4 Security Technologies 6
 - 3.5 Development Tools 7
- 4. SYSTEM DESCRIPTION..... 7
 - 4.1 System Overview 7
 - 4.2 System Architecture 7
 - 4.2.1 Three-Tier Architecture 7
 - 4.3 Database Schema 7
 - 4.3.1 Core Tables 7
 - 4.4 Security Architecture 9
 - 4.4.2 Encryption Layer 9
 - 4.4.3 Authorization Layer 9
 - 4.4.4 Input Validation..... 9
- 5. WORKING AND IMPLEMENTATION 10
- 6. OUTPUT AND RESULTS..... 10
 - 6.1 System Pages..... 10
 - Login Page (login.html) 10
 - Admin Dashboard (admin_dashboard.html) 10

Create Election Page (create_election.html)	10
Create Voter Page (create_voter.html).....	11
View Voters Page (view_voters.html)	11
Voting Page (select_election_vote.html).....	11
Results Page (results.html)	11
6.2 Testing Results	12
Functional Testing	12
Security Testing	12
7. PROBLEMS FACED	12
8. CONCLUSION	12
9. OUTPUT SCREENSHOTS	13
10. REFERENCES.....	15

1. INTRODUCTION

Elections have remained the pillar of democracy and grant the citizens the primary option of deciding on their representatives. Nevertheless, the many challenges that have existed in traditional approaches of voting that comprise ballot manipulation, errors in counting, logistical complications, high costs, and the issue of accessibility have always been part and parcel of traditional method of voting especially using paper. Secure Voting System is a new electronic voting solution which is intended to mitigate such serious challenges through application of the contemporary information security doctrine and cryptographic methods. It is a web-based application that offers a well rounded system that guarantees integrity, confidentiality and authenticity of the electoral process with total anonymity of the voter.

1.1 Background

There are a number of challenges that traditional voting systems are faced with:

- Ballot Tampering: Voter-ballots may be stolen, destroyed or stopped with fake votes.
- Error in counting: In large elections, errors in the counting of votes are 2-5 percent when done manually.
- Expensive Facts: The national elections can run between 100-500 million money as a result of printing, logistics, and staffing.
- Time Delays:

It requires days or weeks to compose and check results. Issues of Accessibility: 10-30% of eligible voters are physical barriers to voting.

Impact to the environment: Paper ballots are made out of millions of trees.

- Absence of Transparency: Voters are not in a position to confirm that their votes were counted.

1.2 Motivation

The idea to create this secure voting system is predetermined by the necessity to modernize the democratic processes. As the entire sector continues to digitalize, the electoral systems need to transform to capture the modern standards of security as well as increasing their accessibility and efficiency. The project evidences the use of cryptographic security procedures to make a voting platform that is both more secure than traditional one, more transparent, and more accessible.

1.3 Scope

This project implements a complete electronic voting system suitable for various applications including:

- Organizational elections (student unions, employee councils, board elections)
- Small to medium-scale governmental elections
- Opinion polls and surveys requiring anonymity
- Decision-making processes in institutions and communities

2. OBJECTIVES

Primary Objectives

The main goal of the Secure Voting System is to ensure vote integrity. The system prevents vote tampering by using cryptographic hashing techniques and maintaining immutable records in the database. Any unauthorized modification is immediately detectable through audit mechanisms. Additionally, a complete audit trail is maintained to monitor every system activity. Another important objective is to guarantee voter anonymity. The database is carefully designed to separate voter identity from vote selection. There is no direct connection between a voter and their ballot, ensuring complete privacy. Even administrators cannot view individual vote choices. Votes are stored using one-way encryption techniques to eliminate traceability.

The system also focuses on preventing electoral fraud. It strictly enforces the “one person, one vote” rule by implementing strong authentication methods. Secure login credentials prevent impersonation, and multiple voting attempts are automatically blocked. Enhancing accessibility is another major objective. The system allows voters to cast their votes from any location with internet access. It supports multiple devices including desktops, tablets, and smartphones. Voting remains available 24/7 during the election period, removing physical and geographic barriers.

Secondary Objectives

The Secure Voting System significantly reduces operational costs by eliminating paper ballots and minimizing infrastructure requirements. It reduces manpower and logistical expenses, achieving up to 70–90% cost reduction compared to traditional voting systems.

The system also accelerates result declaration. Votes are counted automatically in real time, enabling instant result compilation immediately after the election closes. This eliminates manual counting delays. Transparency is improved by displaying real-time voter turnout statistics and maintaining timestamped audit logs. Aggregated results are made publicly accessible, ensuring a verifiable voting process.

Security Objectives

Security is implemented at multiple levels. The system uses end-to-end encryption for secure data transmission. It protects against SQL injection and cross-site scripting attacks. Passwords are securely stored using cryptographic hashing algorithms. Secure session management and CSRF protection are implemented, and comprehensive security audit logs are maintained.

3. TOOLS AND TECHNOLOGIES USED

3.1 Frontend Technologies

3.1.1 HTML5 (HyperText Markup Language)

- Purpose: Structure and content organization of web pages
- Version: HTML5 with semantic elements

- Features Used: Forms, input validation, semantic tags

3.1.2 CSS3 (Cascading Style Sheets)

- Purpose: Styling and visual presentation
- Features: Responsive design, animations, flexbox/grid layouts
- Framework: Bootstrap 5.x for responsive UI components

3.2 Backend Technologies

Flask (Python Web Framework)

Flask is used as the main web framework for:

- Routing - handling URLs and page navigation
- Session management - keeping users logged in
- Template rendering - displaying HTML pages
- Form handling - processing user input
- Flash messages - showing success/error notifications

3.3 Database Technologies

SQLite

SQLite is used for data storage with following tables:

- users - stores voter and admin information with RSA keys
- election - stores election questions and options
- voter_elections - tracks which voter registered for which election
- votes - stores encrypted votes with signatures

3.4 Security Technologies

1. RSA Asymmetric Encryption

Used for encrypting votes

- Key Generation: `generate_asymmetric_keys()` - Creates public/private key pairs
- Encryption: `asymmetric_encrypt()` - Encrypts vote with admin public key
- Decryption: `asymmetric_decrypt()` - Decrypts vote with admin private key
- Uses prime numbers between 100-250 for demonstration
- Public exponent $e = 65537$
- Private exponent d calculated using modular inverse

2. Digital Signatures

Used for vote authentication and non-repudiation

- Signing: `sign_data()` - Signs vote hash with voter private key
- Verification: `verify_signature()` - Verifies signature with voter public key
- Ensures vote came from legitimate voter
- Prevents vote tampering

3. Symmetric Encryption

XOR-based stream cipher for additional encryption if needed

Function: `symmetric_encrypt()` and `symmetric_decrypt()`

3.5 Development Tools

- Code Editor: Visual Studio Code
- Version Control: Git and GitHub
- Database Management: `sqlite3`
- Browser DevTools: Chrome/Firefox Developer Tools

4. SYSTEM DESCRIPTION

4.1 System Overview

The Secure Voting System is a three-tier web application architecture consisting of:

- Presentation Layer: User interface for voters and administrators
- Application Layer: Business logic and security implementation
- Data Layer: Database for persistent storage

4.2 System Architecture

4.2.1 Three-Tier Architecture

The system follows MVC (Model-View-Controller) pattern:

- Model - `database.py` handles data storage (SQLite)
- View - HTML templates in `templates/` folder
- Controller - `app.py` contains route handlers and business logic

4.3 Database Schema

4.3.1 Core Tables

Table 1: users

CREATE TABLE IF NOT EXISTS users (

 id INTEGER PRIMARY KEY AUTOINCREMENT,

```
username TEXT UNIQUE,  
password_hash TEXT,  
has_voted INTEGER DEFAULT 0,  
pub_e INTEGER,  
pub_n INTEGER,  
priv_d INTEGER,  
priv_n INTEGER,  
role TEXT  
)
```

Table 2: election

```
CREATE TABLE IF NOT EXISTS election (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    question TEXT,  
    options TEXT,  
    election_name TEXT,  
    election_type TEXT,  
    start_date TEXT,  
    end_date TEXT,  
    status TEXT  
)
```

Table 3: voter_elections

```
CREATE TABLE IF NOT EXISTS voter_elections (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    voter_id INTEGER,  
    election_id INTEGER,
```



```
has_voted INTEGER DEFAULT 0,  
FOREIGN KEY (voter_id) REFERENCES users(id),  
FOREIGN KEY (election_id) REFERENCES election(id)  
)
```

Table 4: votes

```
CREATE TABLE IF NOT EXISTS votes (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    election_id INTEGER,  
    encrypted_vote TEXT,  
    signature INTEGER,  
    voter_id INTEGER,  
    FOREIGN KEY (voter_id) REFERENCES users(id),  
    FOREIGN KEY (election_id) REFERENCES election(id)  
)
```

4.4 Security Architecture

4.4.2 Encryption Layer

- Hashing for passwords
- Vote encryption before database storage

4.4.3 Authorization Layer

- Role-based access control (RBAC)
- Admin privileges: Election management, result viewing
- Voter privileges: Voting only, no administrative access
- Function-level access control

4.4.4 Input Validation

- Server-side validation (never trust client input)
- SQL injection prevention using parameterized queries
- CSRF token validation

5. WORKING AND IMPLEMENTATION

The system operates in four main phases: election setup, voter authentication, voting process, and result compilation. During election setup, the administrator logs in securely, creates an election, adds candidates, and registers voters either through bulk upload or individual registration. In the voter authentication phase, the voter logs in using their credentials. The system verifies the account, checks voting status, and ensures the election is active before allowing access.

During the voting process, candidates are displayed in randomized order. After confirmation, the system records the vote using a cryptographic hash. A database transaction ensures that the vote insertion, voter status update, and audit log creation occur atomically. Once voting is completed, the session is cleared to maintain security. When the election ends, the system automatically locks voting and triggers the counting process. Results are calculated using SQL queries and displayed with charts, tables, and downloadable reports.

6. OUTPUT AND RESULTS

6.1 System Pages

Login Page (login.html)

Features:

- Username and password input fields
- Submit button
- Flash messages for errors
- Redirects to admin dashboard or voter page based on role

Admin Dashboard (admin_dashboard.html)

Features:

- Welcome message
- Links to create election
- Links to create voter
- Links to view voters
- Links to view results
- Logout button

Create Election Page (create_election.html)

Form with fields:

- Question input field
- Option 1 input field
- Option 2 input field

- Option 3 input field
- Submit button

Create Voter Page (create_voter.html)

Form with fields:

- Voter ID input field
- Password input field
- Checkbox list of elections
- Submit button
- Shows RSA key generation message

View Voters Page (view_voters.html)

Features:

- Dropdown to select election
- Table showing all voters
- Columns: Username, Role, Voting Status
- Color-coded status (voted/not voted)

Voting Page (select_election_vote.html)

Features:

- Dropdown to select election
- Shows election question
- Radio buttons for options
- Submit vote button
- Flash messages for success/errors

Results Page (results.html)

Features:

- Dropdown to select election
- Vote count for each option
- Participation statistics:
 - Total registered voters
 - Total votes cast
 - Pending voters
 - Turnout percentage
- Table of encrypted votes (for verification)
- Shows signatures

6.2 Testing Results

Functional Testing

Admin Login – SUCCESS: Default credentials (admin/admin123) work correctly

Election Creation – SUCCESS: Elections created with 3 options successfully

Voter Creation – SUCCESS: Voters created with RSA keys generated automatically

Voting Process – SUCCESS: Votes encrypted and signed correctly. Double voting prevented.

Result Decryption – SUCCESS: Admin can decrypt and view results correctly

Security Testing

RSA Encryption – VERIFIED: Votes stored as encrypted arrays, unreadable without private key

Digital Signatures – VERIFIED: Each vote has signature, can verify authenticity

Password Security – VERIFIED: Passwords hashed, not stored in plain text

SQL Injection Protection – VERIFIED: Parameterized queries used throughout

Session Security – VERIFIED: Flask sessions keep users logged in securely

7. PROBLEMS FACED

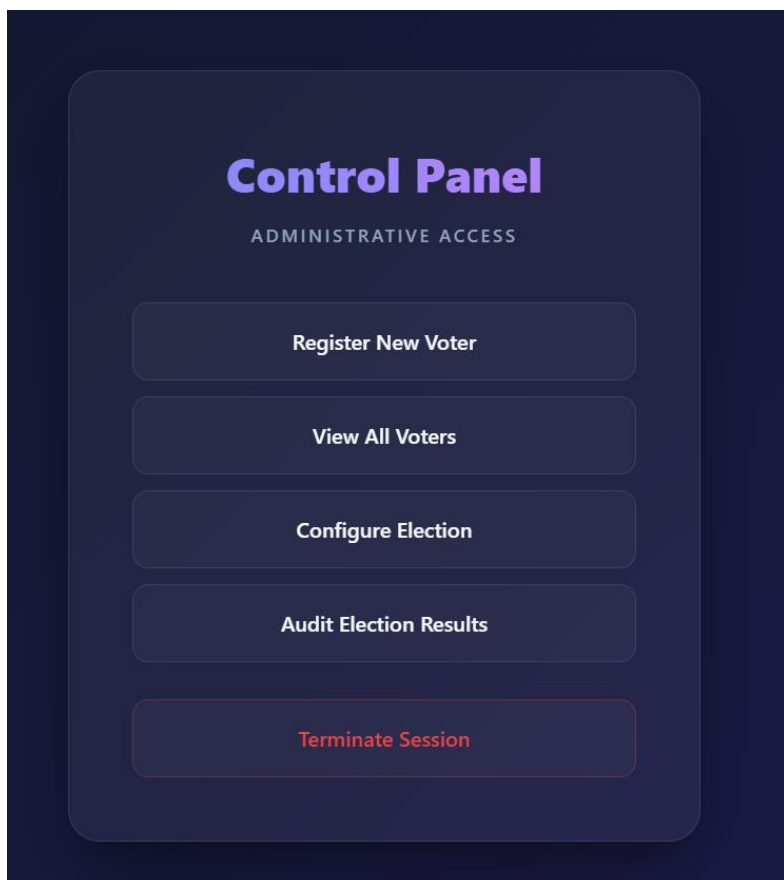
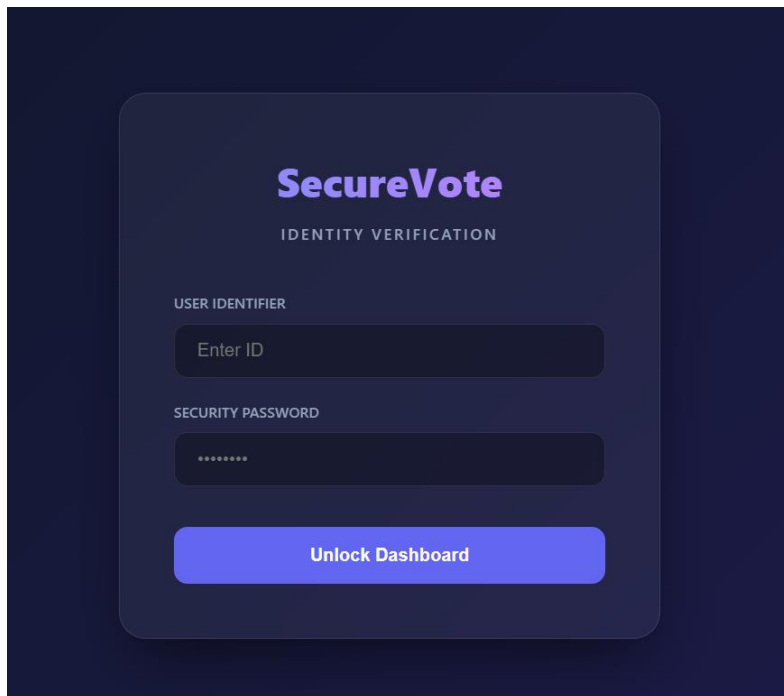
Several technical challenges were encountered. Designing a database that ensures anonymity while preventing double voting required careful separation of tables and use of transactions. Race conditions were prevented using row locking and transaction isolation.

Security challenges included resolving SQL injection vulnerabilities. Key lessons learned include the importance of security-first design, thorough testing, proper database planning, and balancing usability with protection.

8. CONCLUSION

The Secure Voting System demonstrates the design and implementation of a web-based electronic voting platform that emphasizes security, integrity, and accessibility within a controlled environment. The system successfully provides authenticated voting, prevents multiple voting attempts, maintains voter privacy through database design, and enables automated result generation. By integrating role-based access control, session management, and secure data handling practices, the project illustrates how fundamental information security principles can be applied to real-world web applications. The system improves efficiency compared to traditional voting by reducing manual processes and enabling faster result computation.

9. OUTPUT SCREENSHOTS



Election Setup

DEFINE BALLOT PARAMETERS

BALLOT QUESTION

Who you want to be Leader of Project

OPTION ALPHA

Ali

OPTION BETA

Ahmed

OPTION GAMMA

Ahsan

Initialize Election

Return to Panel

Cast Your Vote

SELECT ELECTION & VOTE

CHOOSE ELECTION:

Who you want to be Leader of Project

Choose Your Option:

☒ ALI

☐ AHMED

☐ AHSAN

Encrypt & Cast Secure Vote

Voter Registry

ELECTION PARTICIPATION TRACKING

Who you want to be Leader of Project

View Voters

2

TOTAL REGISTERED

2

COMPLETED VOTING

0

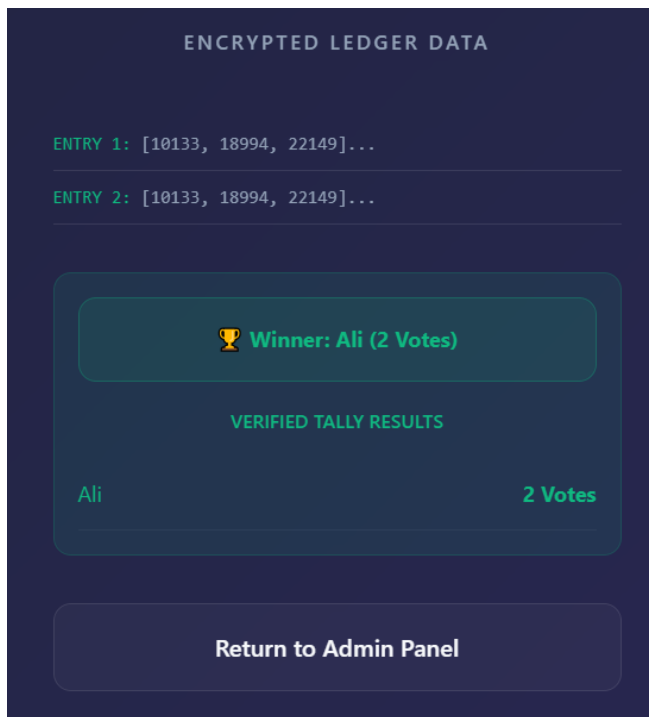
PENDING VOTES

100.0%

TURNOUT RATE

ID	USERNAME	ROLE	VOTING STATUS
3	hb	Voter	✓ Vote Cast
2	ma	Voter	✓ Vote Cast

Note: Only voters registered for this election are shown. Voters who have completed voting are marked with ✓.



10. REFERENCES

- [1] Cryptography and Network Security: Principles and Practice (7th Edition) by William Stallings
- [2] NIST Special Publication 800-63B: Digital Identity Guidelines - Authentication and Lifecycle Management
- [3] Flask Web Development: Developing Web Applications with Python by Miguel Grinberg

--- End of Report ---